2.6 BIVARIATE DATA AND MULTIVARIATE DATA

Bivariate Data involves two variables. Bivariate data deals with causes of relationships. The aim is to find relationships among data. Consider the following Table 2.3, with data of the temperature in a shop and sales of sweaters.

Table 2.3: Temperature in a Shop and Sales Data

Temperature (in centigrade)	Sales of Sweaters (in thousands)
5	200
10	150
15	140
20	75
22	60
23	55
25	20

VtuCircle

Here, the aim of bivariate analysis is to find relationships among variables. The relationships can then be used in comparisons, finding causes, and in further explorations. To do that, graphical display of the data is necessary. One such graph method is called scatter plot.

Scatter plot is used to visualize bivariate data. It is useful to plot two variables with or without nominal variables, to illustrate the trends, and also to show differences. It is a plot between explanatory and response variables. It is a 2D graph showing the relationship between two variables.

The scatter plot (Refer Figure 2.11) indicates strength, shape, direction and the presence of Outliers. It is useful in exploratory data before calculating a correlation coefficient or fitting regression curve.

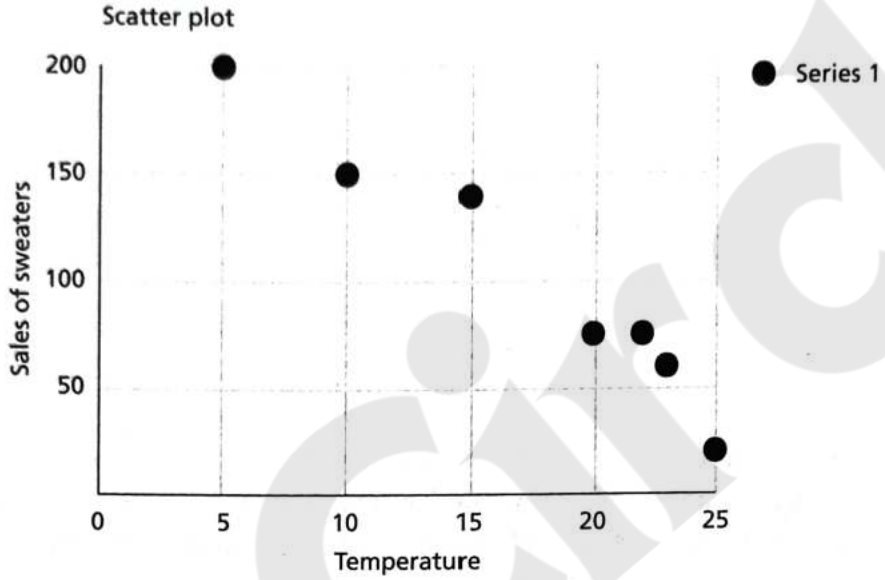

Figure 2.11: Scatter Plot

Line graphs are similar to scatter plots. The Line Chart for sales data is shown in Figure 2.12.

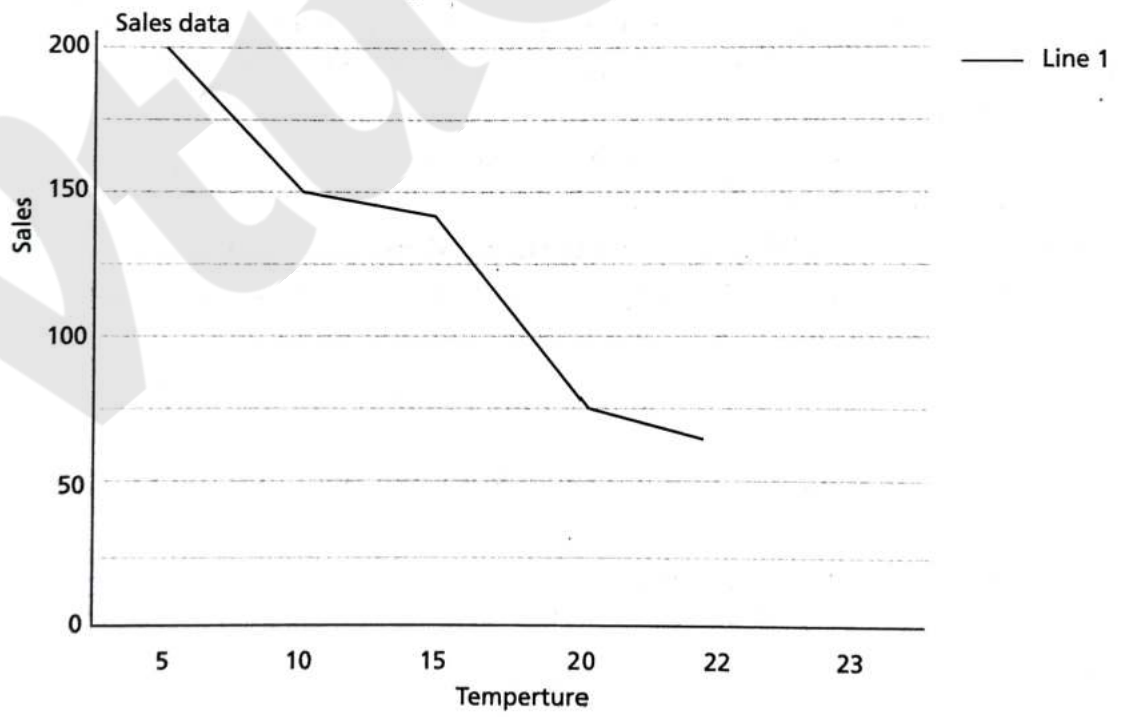

Figure 2.12: Line Chart

2.6.1 Bivariate Statistics

Covariance and Correlation are examples of bivariate statistics. Covariance is a measure of joint probability of random variables, say X and Y. Generally, random variables are represented in capital letters. It is defined as covariance(X, Y) or COV(X, Y) and is used to measure the variance between two dimensions. The formula for finding co-variance for specific x, and y are:

$$\text{cov}(X,Y) = \frac{1}{N}\sum_{i=1}^{N}(x_i - E(X))(y_i - E(Y)) \tag{2.17}$$

Here, x_i and y_i are data values from X and Y. $E(X)$ and $E(Y)$ are the mean values of x_i and y_i. N is the number of given data. Also, the COV(X, Y) is same as COV(Y, X).

Example 2.6: Find the covariance of data $X = \{1, 2, 3, 4, 5\}$ and $Y = \{1, 4, 9, 16, 25\}$.

Solution: Mean(X) = $E(X) = \frac{15}{5} = 3$, Mean(Y) = $E(Y) = \frac{55}{5} = 11$. The covariance is computed using Eq. (2.17) as:

$$\frac{(1-3)(1-11) + (2-3)(4-11) + (3-30)(9-11) + (4-3)(16-11) + (5-3)(25-11)}{5} = 12$$

The covariance between X and Y is 12. It can be normalized to a value between –1 and +1. This is done by dividing it by the correlation of variables. This is called Pearson correlation coefficient. Sometimes, $N - 1$ is also can be used instead of N. In that case, the covariance is 60/4 = 15.

Correlation

The Pearson correlation coefficient is the most common test for determining any association between two phenomena. It measures the strength and direction of a *linear* relationship between the x and y variables.

The correlation indicates the relationship between dimensions using its sign. The sign is more important than the actual value.

1. If the value is positive, it indicates that the dimensions increase together.
2. If the value is negative, it indicates that while one-dimension increases, the other dimension decreases.
3. If the value is zero, then it indicates that both the dimensions are independent of each other.

If the dimensions are correlated, then it is better to remove one dimension as it is a redundant dimension.

If the given attributes are $X = (x_1, x_2, \cdots, x_N)$ and $Y = (y_1, y_2, \cdots, y_N)$, then the Pearson correlation coefficient, that is denoted as r, is given as:

$$r = \frac{COV(X,Y)}{\sigma_X \sigma_Y} \tag{2.18}$$

where, σ_X, σ_Y are the standard deviations of X and Y.

Example 2.7: Find the correlation coefficient of data $X = \{1, 2, 3, 4, 5\}$ and $Y = \{1, 4, 9, 16, 25\}$.

Solution: The mean values of X and Y are $\frac{15}{5} = 3$ and $\frac{55}{5} = 11$. The standard deviations of X and Y are 1.41 and 8.6486, respectively. Therefore, the correlation coefficient is given as ratio of covariance (12 from the previous problem 2.5) and standard deviation of x and y as per Eq. (2.18) as:

$$r = \frac{12}{1.41 \times 8.6486} \approx 0.984$$

2.7 MULTIVARIATE STATISTICS

In machine learning, almost all datasets are multivariable. Multivariate data is the analysis of more than two observable variables, and often, thousands of multiple measurements need to be conducted for one or more subjects.

The multivariate data is like bivariate data but may have more than two dependant variables. Some of the multivariate analysis are regression analysis, principal component analysis, and path analysis.

Id	*Attribute* 1	*Attribute* 2	*Attribute* 3
1	1	4	1
2	2	5	2
3	3	6	1

The mean of multivariate data is a mean vector and the mean of the above three attributes is given as (2, 7.5, 1.33). The variance of multivariate data becomes the covariance matrix. The mean vector is called centroid and variance is called dispersion matrix. This is discussed in the next section.

Multivariate data has three or more variables. The aim of the multivariate analysis is much more. They are regression analysis, factor analysis and multivariate analysis of variance that are explained in the subsequent chapters of this book.

Heatmap

Heatmap is a graphical representation of 2D matrix. It takes a matrix as input and colours it. The darker colours indicate very large values and lighter colours indicate smaller values. The advantage of this method is that humans perceive colours well. So, by colour shaping, larger values can be perceived well. For example, in vehicle traffic data, heavy traffic regions can be differentiated from low traffic regions through heatmap.

In Figure 2.13, patient data highlighting weight and health status is plotted. Here, X-axis is weights and Y-axis is patient counts. The dark colour regions highlight patients' weights vs patient counts in health status.

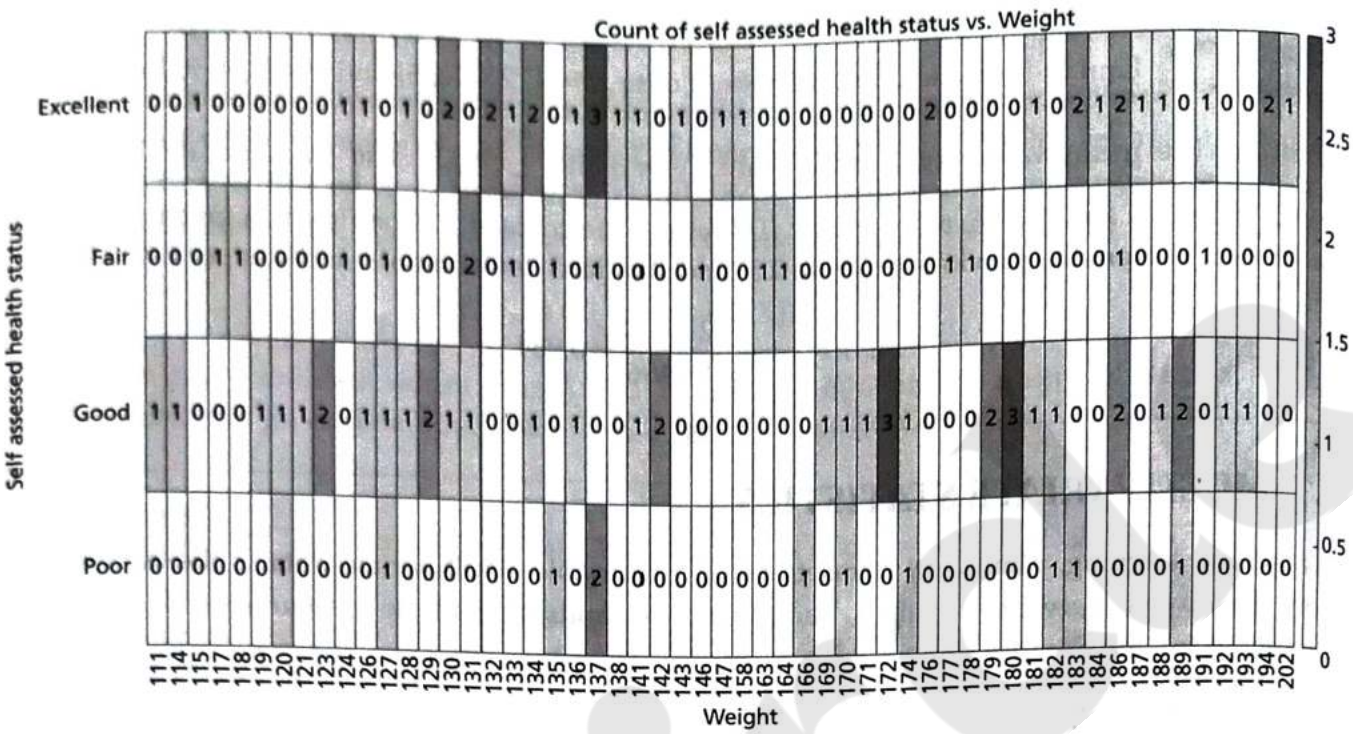

Figure 2.13: Heatmap for Patient Data

Pairplot

Pairplot or scatter matrix is a data visual technique for multivariate data. A scatter matrix consists of several pair-wise scatter plots of variables of the multivariate data. All the results are presented in a matrix format. By visual examination of the chart, one can easily find relationships among the variables such as correlation between the variables.

A random matrix of three columns is chosen and the relationships of the columns is plotted as a pairplot (or scattermatrix) as shown below in Figure 2.14.

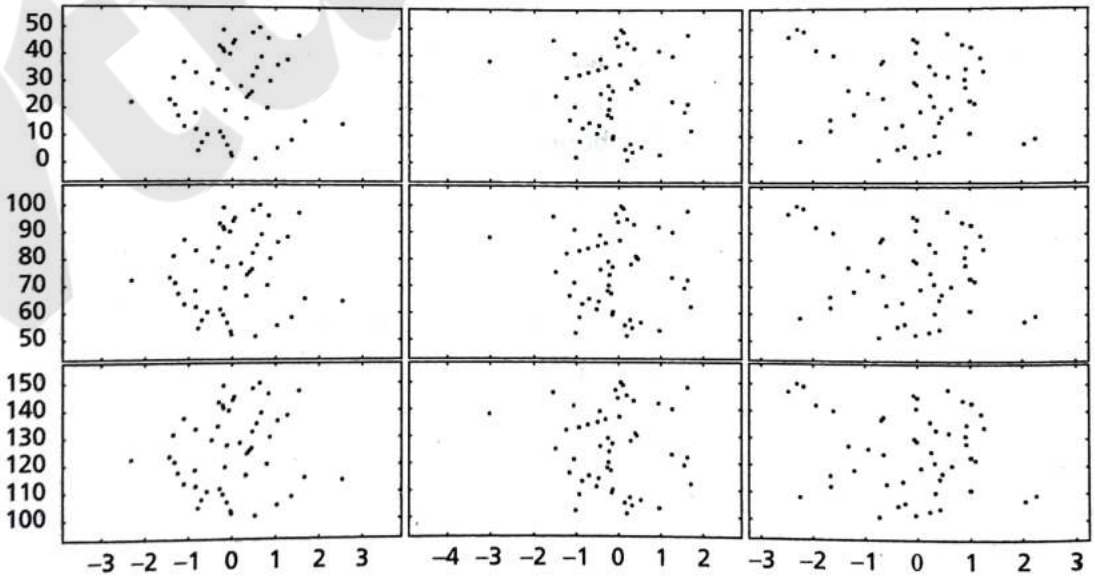

Figure 2.14: Pairplot for Random Data

2.8 ESSENTIAL MATHEMATICS FOR MULTIVARIATE DATA

Machine learning involves many mathematical concepts from the domain of Linear algebra, Statistics, Probability and Information theory. The subsequent sections discuss important aspects of linear algebra and probability.

'Linear Algebra' is a branch of mathematics that is central for many scientific applications and other mathematical subjects. While all branches of mathematics are crucial for machine learning, linear algebra plays a major large role as it is the mathematics of data. Linear algebra deals with linear equations, vectors, matrices, vector spaces and transformations. These are the driving forces of machine learning and machine learning cannot exist without these data types.

Let us discuss some of the important concepts of linear algebra now.

2.8.1 Linear Systems and Gaussian Elimination for Multivariate Data

A linear system of equations is a group of equations with unknown variables.

Let $Ax = y$, then the solution x is given as:

$$x = y/A = A^{-1} y \tag{2.19}$$

This is true if y is not zero and A is not zero. The logic can be extended for N-set of equations with 'n' unknown variables.

It means if $A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ \vdots & \vdots & \vdots & \vdots \\ a_{1n} & a_{2n} & \cdots & a_{nn} \end{pmatrix}$ and $y = (y_1 \ y_2 \cdots y_n)$, then the unknown variable x can be computed as:

$$x = y/A = A^{-1} y \tag{2.20}$$

If there is a unique solution, then the system is called consistent independent. If there are various solutions, then the system is called consistent dependant. If there are no solutions and if the equations are contradictory, then the system is called inconsistent.

For solving large number of system of equations, Gaussian elimination can be used. The procedure for applying Gaussian elimination is given as follows:

1. Write the given matrix.
2. Append vector y to the matrix A. This matrix is called augmentation matrix.
3. Keep the element a_{11} as pivot and eliminate all a_{11} in second row using the matrix operation, $R_2 - \left(\frac{a_{21}}{a_{11}}\right)$, here R_2 is the second row and $\left(\frac{a_{21}}{a_{11}}\right)$ is called the multiplier. The same logic can be used to remove a_{11} in all other equations.
4. Repeat the same logic and reduce it to reduced echelon form. Then, the unknown variable as:

$$x_n = \frac{y_{nn}}{a_{nn}} \tag{2.21}$$

5. Then, the remaining unknown variables can be found by back-substitution as:

$$x_{n-1} = \frac{y_{n-1} - a_{n-1} \times x_n}{a_{(n-1)(n-1)}} \tag{2.22}$$

This part is called backward substitution.

To facilitate the application of Gaussian elimination method, the following row operations are applied:

1. Swapping the rows
2. Multiplying or dividing a row by a constant
3. Replacing a row by adding or subtracting a multiple of another row to it

These concepts are illustrated in Example 2.8.

Example 2.8: Solve the following set of equations using Gaussian Elimination method.

$$2x_1 + 4x_2 = 6$$
$$4x_1 + 3x_2 = 7$$

Solution: Rewrite this in matrix form as follows:

$$\begin{pmatrix} 2 & 4 & | & 6 \\ 4 & 3 & | & 7 \end{pmatrix}$$
$$\sim \begin{pmatrix} 2 & 4 & | & 6 \\ 4 & 3 & | & 7 \end{pmatrix} R_1 = \frac{R_1}{2}$$

Apply the transformation by dividing the row 1 by 2. There are no general guidelines of row operations other than reducing the given matrix to row echelon form. The operator ~ means reducing to. The above matrix can further be reduced as follows:

$$\sim \begin{pmatrix} 1 & 2 & | & 3 \\ 4 & 3 & | & 7 \end{pmatrix} R_2 = R_2 - 4R_1$$
$$\sim \begin{pmatrix} 1 & 2 & | & 3 \\ 0 & -5 & | & -5 \end{pmatrix} R_2 = R_2 / -5$$
$$\sim \begin{pmatrix} 1 & 2 & | & 3 \\ 0 & 1 & | & 1 \end{pmatrix} R_1 = R_1 - 2R_2$$
$$\sim \begin{pmatrix} 1 & 0 & | & 1 \\ 0 & 1 & | & 1 \end{pmatrix}$$

Therefore, in the reduced echelon form, it can be observed that:

$$x_2 = 1$$
$$x_1 = 1$$

2.8.2 Matrix Decompositions

It is often necessary to reduce a matrix to its constituent parts so that complex matrix operations can be performed. These methods are also known as matrix factorization methods.

The most popular matrix decomposition is called eigen decomposition. It is a way of reducing the matrix into eigen values and eigen vectors.

Then, the matrix A can be decomposed as:

$$A = Q \Lambda Q^T \tag{2.23}$$

where, Q is the matrix of eigen vectors, Λ is the diagonal matrix and Q^T is the transpose of matrix Q.

Vtucircle

LU Decomposition

One of the simplest matrix decompositions is *LU* decomposition where the matrix *A* can be decomposed matrices:

$$A = LU$$

Here, *L* is the lower triangular matrix and *U* is the upper triangular matrix. The decomposition can be done using Gaussian elimination method as discussed in the previous section. First, an identity matrix is augmented to the given matrix. Then, row operations and Gaussian elimination is applied to reduce the given matrix to get matrices *L* and *U*.

Example 2.9 illustrates the application of Gaussian elimination to get *LU*.

Example 2.9: Find *LU* decomposition of the given matrix:

$$A = \begin{pmatrix} 1 & 2 & 4 \\ 3 & 3 & 2 \\ 3 & 4 & 2 \end{pmatrix}$$

Solution: First, augment an identity matrix and apply Gaussian elimination. The steps are as shown in:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} 1 & 2 & 4 \\ 3 & 3 & 2 \\ 3 & 4 & 2 \end{bmatrix} \quad \boxed{\text{Initial Matrix}}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 3 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} 1 & 2 & 4 \\ 0 & -3 & -10 \\ 3 & 4 & 2 \end{bmatrix} \quad \boxed{R_2 = R_2 - 3R_1}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 3 & 1 & 0 \\ 3 & 0 & 1 \end{bmatrix}\begin{bmatrix} 1 & 2 & 4 \\ 0 & -3 & -10 \\ 0 & -2 & -10 \end{bmatrix} \quad \boxed{R_3 = R_3 - 3R_1}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 3 & 1 & 0 \\ 3 & \frac{2}{3} & 1 \end{bmatrix}\begin{bmatrix} 1 & 2 & 4 \\ 0 & -3 & -10 \\ 0 & 0 & \frac{-10}{3} \end{bmatrix} \quad \boxed{R_3 = R_3 - \frac{2}{3}R_2}$$

Now, it can be observed that the first matrix is *L* as it is the lower triangular matrix whose values are the determiners used in the reduction of equations above such as 3, 3 and 2/3. The second matrix is *U*, the upper triangular matrix whose values are the values of the reduced matrix because of Gaussian elimination.

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 3 & 1 & 0 \\ 3 & \frac{2}{3} & 1 \end{pmatrix} \text{ and } U = \begin{pmatrix} 1 & 2 & 4 \\ 0 & -3 & -10 \\ 0 & 0 & -\frac{10}{3} \end{pmatrix}.$$

Vtucircle

It can be cross verified that the multiplication of LU yields the original matrix A. What are the applications of LU decomposition? Some of the applications are finding matrix inverses and determinant. If the order of the matrix is large, then this method can be used.

2.8.3 Machine Learning and Importance of Probability and Statistics

Machine learning is linked with statistics and probability. Like linear algebra, statistics is the heart of machine learning. The importance of statistics needs to be stressed as without statistics; analysis of data is difficult. Probability is especially important for machine learning. Any data can be assumed to be generated by a probability distribution. Machine learning datasets have multiple data that are generated by multiple distributions. So, a knowledge of probability distribution and random variables are must for better understanding of the machine learning concepts.

What is the objective for an experiment? This is about hypothesis or constructing a model. Machine learning has many models. So, the theory of hypothesis, construction of models, evaluating the models using hypothesis testing, significance of the model and evaluation of the model are all linking factors of machine learning and probability theory. Similarly, the construction of datasets using sampling theory is also required.

The subsequent section will deal with the important concepts of statistics and probability.

Probability Distributions

A probability distribution of a variable, say X, summarizes the probability associated with X's events. Distribution is a parameterized mathematical function. In other words, distribution is a function that describes the relationship between the observations in a sample space.

Consider a set of data. The data is said to follow a distribution if it obeys a mathematical function that characterizes that distribution. The function can be used to calculate the probability of individual observations.

Probability distributions are of two types:

1. Discrete probability distribution
2. Continuous probability distribution

The relationships between the events for a continuous random variable and their probabilities is called a continuous probability distribution. It is summarized as Probability Density Function (PDF). PDF calculates the probability of observing an instance. The plot of PDF shows the shape of the distribution. Cumulative Distributive Function (CDF) computes the probability of an observation ≤ value.

Both PDF and CDF are continuous values. The discrete equivalent of PDF in discrete distribution is called Probability Mass Function (PMF).

The probability of an event cannot be detected directly. It should be computed as the area under the curve for a small interval around the specific outcome. This is defined as CDF.

Let us discuss some of the distributions that are encountered in machine learning.

Continuous Probability Distributions Normal, Rectangular, and Exponential distributions fall under this category.

1. Normal Distribution – Normal distribution is a continuous probability distribution. This is also known as gaussian distribution or bell-shaped curve distribution. It is the most common distribution function. The shape of this distribution is a typical bell-shaped curve. In normal distribution, data tends to be around a central value with no bias on left or right. The heights of the students, blood pressure of a population, and marks scored in a class can be approximated using normal distribution.

   PDF of the normal distribution is given as:

$$f(x, \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \tag{2.24}$$

   Here, μ is mean and σ is the standard deviation. Normal distribution is characterized by two parameters – mean and variance.

   Mostly, one uses the normal distribution curve of mean 0 and a *SD* of 1. In normal distribution, mean, median and mode are same. The distribution extends from $-\infty$ to $+\infty$. Standard deviation is how the data is spread out.

   One important concept associated with normal distribution is *z*-score. It can be computed as:

   $z = \frac{x - \mu}{\sigma}$. When μ is zero and σ is 1, *z*-score is same as x. This is useful to normalize the data.

   Most of the statistical tests expect data to follow normal distribution. To check it, normality tests are used. Normality test of the data can be done by Q-Q plot where CDF of one random variable follows CDF of normal distribution. Then, quantity of one distribution is plotted against other distributions. If they are same, then the plot closely follows the straight line from bottom-left to top-right.

2. Rectangular Distribution – This is also known as uniform distribution. It has equal probabilities for all values in the range a, b. The uniform distribution is given as follows:

$$P(X = x) = \begin{cases} \frac{1}{b-a} & \text{for } a \le x \le b \\ 0 & \text{Otherwise} \end{cases} \tag{2.25}$$

3. Exponential Distribution – This is a continuous uniform distribution. This probability distribution is used to describe the time between events in a Poisson process. Exponential distribution is another special case of Gamma distribution with a fixed parameter of 1. This distribution is helpful in modelling of time until an event occurs.

   The PDF is given as follows:

$$f(x, \lambda) = \begin{cases} \lambda e^{-\lambda x} & \text{if } x \ge 0 \quad (\lambda > 0) \\ 0 & \text{if } x < 0 \end{cases} \tag{2.26}$$

   Here, x is a random variable and λ is called rate parameter. The mean and standard deviation of exponential distribution is given as β, where, $\beta = \frac{1}{\lambda}$.

Discrete Distribution Binomial, Poisson, and Bernoulli distributions fall under this category.

1. Binomial Distribution – Binomial distribution is another distribution that is often encountered in machine learning. It has only two outcomes: success or failure. This is also called Bernoulli trial.

   The objective of this distribution is to find probability of getting success k out of n trials. The way to get success out of k out of n number of trials is given as:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} \tag{2.27}$$

   The binomial distribution function is given as follows, where p is the probability of success and probability of failure is $(1-p)$. The probability of success in a certain number of trials is given as:

$$p^k(1-p)^{n-k} \text{ or } p^k q^{n-k} \tag{2.28}$$

   Combining both, one gets PDF of binomial distribution as:

$$\binom{n}{k} p^k(1-p)^{n-k} \tag{2.29}$$

   Here, p is the probability of each choice, k is the number of choices, and n is the total number of choices. The mean of binomial distribution is given below:

$$\mu = n \times p \tag{2.30}$$

   And the variance is given as:

$$\sigma^2 = np(1-p) \tag{2.31}$$

   Hence, the standard deviation is given as:

$$\sigma = \sqrt{np(1-p)} \tag{2.32}$$

2. Poisson Distribution – It is another important distribution that is quite useful. Given an interval of time, this distribution is used to model the probability of a given number of events k. The mean rule λ is inclusive of previous events. Some of the examples of Poisson distribution are number of emails received, number of customers visiting a shop and the number of phone calls received by the office.

   The PDF of Poisson distribution is given as follows:

$$f(X = x; \lambda) = Pr[X = x] = \frac{e^{-\lambda}\lambda^x}{x!} \tag{2.33}$$

   Here, x is the number of times the event occurs and λ is the mean number of times an event occurs.

   The mean is the population mean at number of emails received and the standard deviation is $\sqrt{\lambda}$.

3. Bernoulli Distribution – This distribution models an experiment whose outcome is binary. The outcome is positive with p and negative with $1-p$. The PMF of this distribution is given as:

$$f(k;p) = \begin{cases} q = 1-p & \text{if } k = 0 \\ p & \text{if } k = 1. \end{cases} \tag{2.34}$$

   The mean is p and variance is $p(1-p) = q$

Density Estimation

Let there be a set of observed values $x_1, x_2, \cdots, x_n$ from a larger set of data whose distribution is not known. Density estimation is the problem of estimating the density function from an observed data. The estimated density function, denoted as, $p(x)$ can be used to value directly for any unknown data, say x_t as $p(x_t)$. If its value is less than ε, then x_t is not an outlier or anomaly data. Else, it is categorized as an anomaly data.

There are two types of density estimation methods, namely parametric density estimation and non-parametric density estimation.

Parametric Density Estimation It assumes that the data is from a known probabilistic distribution and can be estimated as $p(x \mid \Theta)$, where, Θ is the parameter. Maximum likelihood function is a parametric estimation method.

Maximum Likelihood Estimation For a sample of observations, one can estimate the probability distribution. This is called density estimation. Maximum Likelihood Estimation (MLE) is a probabilistic framework that can be used for density estimation. This involves formulating a function called likelihood function which is the conditional probability of observing the observed samples and distribution function with its parameters. For example, if the observations are $X = \{x_1, x_2, \cdots, x_n\}$, then density estimation is the problem of choosing a PDF with suitable parameters to describe the data. MLE treats this problem as a search or optimization problem where the probability should be maximized for the joint probabilities of X and its parameter, θ.

For example, this is expressed as $p(X; \theta)$, where, $X = \{x_1, x_2, \cdots, x_n\}$

The likelihood of observing the data is given as a function $L(X; \theta)$. The objective of MLE is to maximize this function as $max\ L(X; \theta)$.

The joint probability of this problem can be restated as $\prod_{i=1}^{n} p(x_i; \theta)$.

The computation of the above formula is unstable and the hence the problem is restated as maximum of log conditional probability given θ. This is given as:

$$\sum_{i=1}^{n} \log p(x_i; \theta) \tag{2.35}$$

Instead of maximizing, one can minimize this function as:

$min = -\sum_{i=1}^{n} \log p(x_i; \theta)$ as, often, minimization is preferred over maximization.

This is called negative log-likelihood function.

The relevance of this theory of MLE for machine learning is that MLE can solve the problem of predictive modelling in machine learning. Regression problem is treated in Chapter 5 using the principle of least-square approach. The same problem can be viewed from MLE perspective too. If one assumes that the regression problem can be framed as predicting output y given input x, then for $p(y/x)$, the MLE framework can be applied as:

$$max \sum \log(y \mid x_i, h) \tag{2.36}$$

Here, h is the linear regression model. If Gaussian distribution is assumed as it is an obvious fact that most of the data follow Gaussian distribution, then MLE can be stated as:

$$\max \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{y_i - h(x_i;\beta)}{2\sigma^2}} \tag{2.37}$$

Here, β is the regression coefficient and x_i is the given sample. One can maximize this function or minimize the negative log likelihood function to provide a solution for linear regression problem. The Eq. (2.37) yields the same answer of the least-square approach.

Stochastic gradient descent (SGD) is an algorithm that is normally used to maximize or minimize any function. This algorithm can be used to minimize the above function to get the results effectively. Hence, many statistical packages use this approach for solving linear regression problem. Theoretically, any model can be used instead of linear regression in the above equation. Logistic regression is another model that can be used in MLE framework. Logistic regression is discussed in Chapter 5 of this book.

Gaussian Mixture Model and Expectation-Maximization (EM) Algorithm In machine learning, clustering is one of the important tasks. It is discussed in Chapter 13. MLE framework is quite useful for designing model-based methods for clustering data. A model is a statistical method and data is assumed to be generated by a distribution model with its parameter, θ. There may be many distributions involved and that is why it is called as mixture model. Since, Gaussian are normally assumed for data, this mixture model is categorized as Gaussian Mixture Model (GMM).

The EM algorithm is one algorithm that is commonly used for estimating the MLE in the presence of latent or missing variables. What is a latent variable? Let us assume that the dataset includes weights of boys and girls. Considering the fact that the boys' weights would be slightly more than the weights of the girls, one can assume that the larger weights are generated by one gaussian distribution with one set of parameters while girls' weights are generated with another set of parameters. There is an influence of gender in the data, but it is not directly present or observable. These are called latent variables. The EM algorithm is effective for estimating the PDF in the presence of latent variables.

Generally, there can be many unspecified distributions with different set of parameters. The EM algorithm has two stages:

1. Expectation (E) Stage – In this stage, the expected PDF and its parameters are estimated for each latent variable.
2. Maximization (M) stage – In this, the parameters are optimized using the MLE function.

This process is iterative, and the iteration is continued till all the latent variables are fitted by probability distributions effectively along with the parameters.

Non-parametric Density Estimation A non-parametric estimation can be generative or discriminative. Parzen window is a generative estimation method that finds $p(x \mid \Theta)$ as conditional density. Discriminative methods directly compute $p(\Theta \mid x)$ as posteriori probability. Parzen window and k-Nearest Neighbour (KNN) rule are examples of non-parametric density estimation. Let us discuss about them now.

Parzen Window Let there be 'n' samples, $X = \{x_1, x_2, \cdots, x_n\}$

The samples are drawn independently, called as identically independent distribution. Let R be the region that covers 'k' samples of total 'n' samples. Then, the probability density function is given as:

$$p = k/n \tag{2.38}$$

The estimate is given as:

$$p(x) = \frac{k/n}{V} \tag{2.39}$$

where, V is the volume of the region R. If R is the hypercube centered at x and h is the length of the hypercube, the volume V is h^2 for 2D square cube and h^3 for 3D cube.

The Parzen window is given as follows:

$$\varphi\left(\frac{x_i - x}{h}\right) = \begin{cases} 1 \text{ if } \dfrac{|x_{ik} - x_k|}{h} < \dfrac{1}{2} \\ 0 \text{ otherwise} \end{cases} \tag{2.40}$$

The window indicates if the sample is inside the region or not. The Parzen probability density function estimate using Eq. (2.40) is given as:

$$\begin{aligned} p(x) &= \frac{k/n}{V} \\ &= \frac{1}{n}\sum_{i=1}^{n}\frac{1}{V_n}\varphi\left(\frac{x_i - x}{h}\right) \end{aligned} \tag{2.41}$$

This window can be replaced by any other function too. If Gaussian function is used, then it is called Gaussian density function.

KNN Estimation The KNN estimation is another non-parametric density estimation method. Here, the initial parameter k is determined and based on that k-neighbours are determined. The probability density function estimate is the average of the values that are returned by the neighbours.

Vtucircle

2.10 FEATURE ENGINEERING AND DIMENSIONALITY REDUCTION TECHNIQUES

Features are attributes. Feature engineering is about determining the subset of features that form an important part of the input that improves the performance of the model, be it classification or any other model in machine learning.

Feature engineering deals with two problems – Feature Transformation and Feature Selection. Feature transformation is extraction of features and creating new features that may be helpful in increasing performance. For example, the height and weight may give a new attribute called Body Mass Index (BMI).

Feature subset selection is another important aspect of feature engineering that focuses on selection of features to reduce the time but not at the cost of reliability.

The subset selection reduces the dataset size by removing irrelevant features and constructs a minimum set of attributes for machine learning. If the dataset has n attributes, then time complexity is extremely high as n dimensions need to be processed for the given dataset. For n attributes, there are 2^n possible subsets. If the value of n is high, the problem becomes intractable. This is called 'curse of dimensionality'. Since, as the number of dimensions increases, the time complexity increases. The remedy is that some of the components that do not contribute much can be deleted. This results in the reduction of dimensionality. Choosing optimal attributes becomes a graph search problem. Typically, the feature subset selection problem uses greedy approach by looking for the best choice at the time using locally optimal choice while hoping that it would lead to global optimal solutions.

The features can be removed based on two aspects:

1. Feature relevancy – Some features contribute more for classification than other features. For example, a mole on the face can help in face detection than common features like nose. In simple words, the features should be relevant. The relevancy of the features can be determined based on information measures such as mutual information, correlation-based features like correlation coefficient and distance measures. Distance measures are discussed in Chapter 13 of this book.
2. Feature redundancy – Some features are redundant. For example, when a database table has a field called Date of birth, then age field is not relevant as age can be computed easily from date of birth. This helps in removing the column age that leads to reduction of dimension one.

So, the procedure is:

1. Generate all possible subsets
2. Evaluate the subsets and model performance
3. Evaluate the results for optimal feature selection

Filter-based selection uses statistical measures for assessing features. In this approach, no learning algorithm is used. Correlation and information gain measures like mutual information and entropy are all examples of this approach.

Wrapper-based methods use classifiers to identify the best features. These are selected and evaluated by the learning algorithms. This procedure is computationally intensive but has superior performance.

Let us discuss some of the important algorithms that fall under this category.

2.10.1 Stepwise Forward Selection

This procedure starts with an empty set of attributes. Every time, an attribute is tested for statistical significance for best quality and is added to the reduced set. This process is continued till a good reduced set of attributes is obtained.

2.10.2 Stepwise Backward Elimination

This procedure starts with a complete set of attributes. At every stage, the procedure removes the worst attribute from the set, leading to the reduced set.

Combined Approach Both forward and reverse methods can be combined so that the procedure can add the best attribute and remove the worst attribute.

2.10.3 Principal Component Analysis

The idea of the principal component analysis (PCA) or KL transform is to transform a given set of measurements to a new set of features so that the features exhibit high information packing properties. This leads to a reduced and compact set of features. Basically, this elimination is made possible because of the information redundancies. This compact representation is of a reduced dimension.

Consider a group of random vectors of the form:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

The *mean vector* of the set of random vectors is defined as:

$$m_x = E\{x\}$$

The operator E refers to the expected value of the population. This is calculated theoretically using the probability density functions (PDF) of the elements x_i and the joint probability density functions between the elements x_i and x_j. From this, the covariance matrix can be calculated as:

$$C = E\{(x - m_x)(x - m_x)^T\} \tag{2.52}$$

For M random vectors, when M is large enough, the mean vector and covariance matrix can be approximately calculated as:

$$m_x = \frac{1}{M}\sum_{k=1}^{M} x_k \tag{2.53}$$

$$A = \frac{1}{M}\sum_{k=1}^{M} x_k x_k^T - m_x m_x^T \tag{2.54}$$

This covariance matrix is real and symmetric. If e_i and λ_i (where, $i = 1, 2, \ldots, n$) be the set of eigen vectors and corresponding eigen values of the covariance matrix, the eigen values can be arranged in a descending order so that $\lambda_i \geq \lambda_{i+1}$ for $i = 1, 2, \ldots, n-1$. The corresponding eigen vectors are calculated. Based on this, the transform kernel is constructed. Let the transform kernel be A. Then, the matrix rows are formed from the eigen vectors of the covariance matrix.

The mapping of the vectors x to y using the transformation can now be described as:

$$y = A(x - m_x) \tag{2.55}$$

This transform is also called as Karhunen-Loeve or Hoteling transform. The original vector x can now be reconstructed as follows:

$$x = A^T y + m_x \tag{2.56}$$

The goal of PCA is to reduce the set of attributes to a newer, smaller set that captures the variance of the data. The variance is captured by fewer components, which would give the same result as the original, with all the attributes. Here, instead of using all the eigen vectors of the covariance matrix, only a small set that exhibits the variance can be used. By using a small set, the KL transform can achieve maximum compression of the available data.

If K largest eigen values are used, the recovered information would be:

$$x = A_K^T y + m_x \tag{2.57}$$

The advantages of PCA are immense. It reduces the attribute list by eliminating all irrelevant attributes. The PCA algorithm is as follows:

1. The target dataset x is obtained
2. The mean is subtracted from the dataset. Let the mean be m. Thus, the adjusted dataset is $X - m$. The objective of this process is to transform the dataset with zero mean.
3. The covariance of dataset x is obtained. Let it be C.

4. Eigen values and eigen vectors of the covariance matrix are calculated.
5. The eigen vector of the highest eigen value is the principal component of the dataset. The eigen values are arranged in a descending order. The feature vector is formed with these eigen vectors in its columns.

   Feature vector = {eigen vector$_1$, eigen vector$_2$, ... , eigen vector$_n$}
6. Obtain the transpose of feature vector. Let it be A.
7. PCA transform is $y = A \times (x - m)$, where x is the input dataset, m is the mean, and A is the transpose of the feature vector.

The original data can be retrieved using the formula given below:

$$\text{Original data } (f) = \{(A)^{-1} \times y\} + m \tag{2.58}$$

$$= \{(A)^{T} \times y\} + m \tag{2.59}$$

The new data is a dimensionally reduced matrix that represents the original data. Therefore, PCA is effective in removing the attributes that do not contribute. If the original data is required, it can be obtained with no loss of information. Scree plot is a visualization technique to visualize the principal components or variables that play a more important role as compared to other attributes. Scree plot is a visualization technique to visualize the principal components visually. For a randomly selected dataset of 246 attributes, PCA is applied and its scree plot is shown in Figure 2.15. The scree plot indicates that only 6 out of 246 attributes are important.

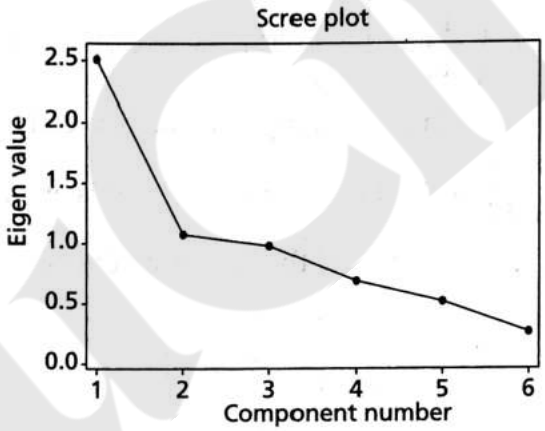

Figure 2.15: Scree Plot

From Figure 2.15, one can infer the relevance of the attributes. The scree plot indicates that the first attribute is more important than all other attributes.

Example 2.12: Let the data points be $\begin{pmatrix}2\\6\end{pmatrix}$ and $\begin{pmatrix}1\\7\end{pmatrix}$. Apply PCA and find the transformed data. Again, apply the inverse and prove that PCA works.

Solution: One can combine two vectors into a matrix as follows:

The mean vector can be computed as Eq. (2.53) as follows:

$$\mu = \begin{pmatrix}\frac{2+1}{2}\\ \frac{6+7}{2}\end{pmatrix} = \begin{pmatrix}1.5\\6.5\end{pmatrix}$$

As part of PCA, the mean must be subtracted from the data to get the adjusted data:

$$x_1 = \begin{pmatrix} 2 - 1.5 \\ 6 - 6.5 \end{pmatrix} = \begin{pmatrix} 0.5 \\ -0.5 \end{pmatrix}$$

$$x_2 = \begin{pmatrix} 1 - 1.5 \\ 7 - 6.5 \end{pmatrix} = \begin{pmatrix} -0.5 \\ 0.5 \end{pmatrix}$$

One can find the covariance for these data vectors. The covariance can be obtained using Eq. (2.54):

$$m_1 = \begin{pmatrix} 0.5 \\ -0.5 \end{pmatrix} \begin{pmatrix} 0.5 & -0.5 \end{pmatrix} = \begin{pmatrix} 0.25 & -0.25 \\ -0.25 & 0.25 \end{pmatrix}$$

$$m_2 = \begin{pmatrix} -0.5 \\ 0.5 \end{pmatrix} \begin{pmatrix} -0.5 & 0.5 \end{pmatrix} = \begin{pmatrix} 0.25 & -0.25 \\ -0.25 & 0.25 \end{pmatrix}$$

The final covariance matrix is obtained by adding these two matrices as:

$$C = \begin{pmatrix} 0.5 & -0.5 \\ -0.5 & 0.5 \end{pmatrix}$$

The eigen values and eigen vectors of matrix C can be obtained (left as an exercise) as $\lambda_1 = 1$, $\lambda_1 = 0$. The eigen vectors are $\begin{pmatrix} -1 \\ 1 \end{pmatrix}$ and $\begin{pmatrix} 1 \\ 1 \end{pmatrix}$. The matrix A can be obtained by packing the eigen vector of these eigen values (after sorting it) of matrix C. For this problem, $A = \begin{pmatrix} -1 & 1 \\ 1 & 1 \end{pmatrix}$. The transpose of A, $A^T = \begin{pmatrix} -1 & 1 \\ 1 & 1 \end{pmatrix}$ is also the same matrix as it is an orthogonal matrix. The matrix can be normalized by diving each elements of the vector, by the norm of the vector to get:

$$A = \begin{pmatrix} -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{pmatrix}$$

One can check that the PCA matrix A is orthogonal. A matrix is orthogonal is $A^{-1} = A$ and $AA^{-1} = I$.

$$AA^T = \begin{pmatrix} -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{pmatrix} \begin{pmatrix} -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{pmatrix}$$

$$= \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

The transformed matrix y using Eq. (2.55) is given as:

$$y = A \times (x - m)$$

Recollect that $(x-m)$ is the adjusted matrix.

$$y = A(x-m) = \begin{pmatrix} -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{pmatrix} \begin{pmatrix} 0.5 & -0.5 \\ -0.5 & 0.5 \end{pmatrix}$$

$$= \begin{pmatrix} -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{pmatrix} \begin{pmatrix} \frac{1}{2} & -\frac{1}{2} \\ -\frac{1}{2} & \frac{1}{2} \end{pmatrix} \left(\textit{for convenience } 0.5 = \frac{1}{2} \right)$$

$$= \begin{pmatrix} -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 0 & 0 \end{pmatrix}$$

One can check the original matrix can be retrieved from this matrix as:

$$\{(A)^T \times y\} + m$$

$$x = A^T y + m = \begin{pmatrix} -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{pmatrix} \begin{pmatrix} -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} 1.5 \\ 6.5 \end{pmatrix}$$

$$= \begin{pmatrix} \frac{1}{2} & -\frac{1}{2} \\ -\frac{1}{2} & \frac{1}{2} \end{pmatrix} + \begin{pmatrix} 1.5 \\ 6.5 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 6 & 7 \end{pmatrix}$$

Therefore, one can infer the original is obtained without any loss of information.

2.10.4 Linear Discriminant Analysis

Linear Discriminant Analysis (LDA) is also a feature reduction technique like PCA. The focus of LDA is to project higher dimension data to a line (lower dimension data). LDA is also used to classify the data. Let there be two classes, c_1 and c_2. Let μ_1 and μ_2 be the mean of the patterns of two classes. The mean of the class c_1 and c_2 can be computed as:

$$\mu_1 = \frac{1}{N_1} \sum_{x_i \in c_1}^{n} x_i \text{ and } \mu_2 = \frac{1}{N_2} \sum_{x_i \in c_2}^{n} x_i$$

The aim of LDA is to optimize the function:

$$J(V) = \frac{V^T \sigma_B V}{V^T \sigma_W V} \tag{2.60}$$

where, V is the linear projection and σ_B and σ_W are class scatter matrix and within scatter matrix, respectively. For the two-class problem, these matrices are given as:

$$\sigma_B = N_1(\mu_1 - \mu)(\mu_1 - \mu)^T + N_2(\mu_2 - \mu)(\mu_2 - \mu)^T \tag{2.61}$$

$$\sigma_W = \sum_{x_i \in c_1} (x_i - \mu_1)(x_i - \mu_1)^T + \sum_{x_i \in c_2} (x_i - \mu_2)(x_i - \mu_2)^T \tag{2.62}$$

The maximization of $J(V)$ should satisfy the equation:

$$\sigma_B V = \lambda \sigma_W V \text{ or } \sigma_W^{-1}\sigma_B V = \lambda V \tag{2.63}$$

As $\sigma_B V$ is always in the direction of $(\mu_1 - \mu_2)$, V can be given as:

$$V = \sigma_W^{-1}(\mu_1 - \mu_2) \tag{2.64}$$

Let $V = \{v_1, v_2, \cdots, v_d\}$ be the generalized eigen vectors of σ_B and σ_W, where, d is the largest eigen values as in PCA. The transformation of x is then given as:

$$y = V_d^T x \tag{2.65}$$

Like in PCA, the largest eigen values can be retained to have projections.

2.10.5 Singular Value Decomposition

Singular Value Decomposition (SVD) is another useful decomposition technique. Let A be the matrix, then the matrix A can be decomposed as:

$$A = USV^T \tag{2.66}$$

Here, A is the given matrix of dimension $m \times n$, U is the orthogonal matrix whose dimension is $m \times n$, S is the diagonal matrix of dimension $n \times n$, and V is the orthogonal matrix. The procedure for finding decomposition matrix is given as follows:

1. For a given matrix, find AA^T
2. Find eigen values of AA^T
3. Sort the eigen values in a descending order. Pack the eigen vectors as a matrix U.
4. Arrange the square root of the eigen values in diagonal. This matrix is diagonal matrix, S.
5. Find eigen values and eigen vectors for A^TA. Find the eigen value and pack the eigen vector as a matrix called V.

Thus, $A = USV^T$. Here, U and V are orthogonal matrices. The columns of U and V are left and right singular values, respectively. SVD is useful in compression, as one can decide to retain only a certain component instead of the original matrix A as:

$$a_{ij} = \sum_{k=1}^{n} u_{ik} s_k v_{jk} \tag{2.67}$$

Based on the choice of retention, the compression can be controlled.

Example 2.13: Find SVD of the matrix:

$$A = \begin{pmatrix} 1 & 2 \\ 4 & 9 \end{pmatrix}$$

Solution: The first step is to compute:

$$AA^T = \begin{pmatrix} 1 & 2 \\ 4 & 9 \end{pmatrix}\begin{pmatrix} 1 & 4 \\ 2 & 9 \end{pmatrix} = \begin{pmatrix} 5 & 22 \\ 22 & 97 \end{pmatrix}$$

The eigen value and eigen vector of this matrix can be calculated to get U. The eigen values of this matrix are 0.0098 and 101.9902.

The eigen vectors of this matrix are:

$$u_1 = \begin{pmatrix} 0.2268 \\ 1 \end{pmatrix}$$
$$u_2 = \begin{pmatrix} -4.4086 \\ 1 \end{pmatrix}$$

These vectors are normalized to get the vectors respectively as:

$$u_1 = \begin{pmatrix} 0.2212 \\ 0.9752 \end{pmatrix}$$
$$u_2 = \begin{pmatrix} -0.9752 \\ 0.2212 \end{pmatrix}$$

The matrix U can be obtained by concatenating the above vector as:

$$U = [u_1, u_2] = \begin{pmatrix} 0.2212 & -0.9752 \\ 0.9752 & 0.2212 \end{pmatrix}$$

The matrix V can be obtained by finding A^TA. It is $\begin{pmatrix} 17 & 38 \\ 38 & 85 \end{pmatrix}$. The eigen values are 0.0098 and 101.9902. The eigen vectors can be found as follows:

$$v_1 = \begin{pmatrix} 0.447 \\ 1 \end{pmatrix} \text{ when } \lambda = 101.99$$

$$v_2 = \begin{pmatrix} -2.236 \\ 1 \end{pmatrix} \text{ when } \lambda = 0.0098$$

The above can be normalized as follows:

$$v_1 = \begin{pmatrix} 0.4082 \\ 0.9129 \end{pmatrix}$$
$$v_2 = \begin{pmatrix} -0.9129 \\ 0.4082 \end{pmatrix}$$

The matrix V can be obtained by concatenating the above vector as:

$$V = [v_1 \ \ v_2] = \begin{pmatrix} 0.4081 & -0.9129 \\ 0.9129 & 0.4082 \end{pmatrix}$$

The matrix S can be found as the diagonal matrix as:

$$S = \begin{pmatrix} \sqrt{101.9902} & 0 \\ 0 & \sqrt{0.0098} \end{pmatrix} = \begin{pmatrix} 10.099 & 0 \\ 0 & 0.099 \end{pmatrix}$$

Therefore, the matrix decomposition $A = U\,SV^T$ is complete.

The main advantage of SVD is compression. A matrix, say an image, can be decomposed and selectively only certain components can be retained by making all other elements zero. This reduces the contents of image while retaining the quality of the image. SVD is useful in data reduction too.

VtuCircle

3.3 DESIGN OF A LEARNING SYSTEM

A system that is built around a learning algorithm is called a learning system. The design of systems focuses on these steps:

1. Choosing a training experience
2. Choosing a target function
3. Representation of a target function
4. Function approximation

Training Experience

Let us consider designing of a chess game. In direct experience, individual board states and correct moves of the chess game are given directly. In indirect system, the move sequences and results are only given. The training experience also depends on the presence of a supervisor who can label all valid moves for a board state. In the absence of a supervisor, the game agent plays against itself and learns the good moves, if the training samples cover all scenarios, or in other words, distributed enough for performance computation. If the training samples and testing samples have the same distribution, the results would be good.

Determine the Target Function

The next step is the determination of a target function. In this step, the type of knowledge that needs to be learnt is determined. In direct experience, a board move is selected and is determined whether it is a good move or not against all other moves. If it is the best move, then it is chosen as: B –> M, where, B and M are legal moves. In indirect experience, all legal moves are accepted and a score is generated for each. The move with largest score is then chosen and executed.

Determine the Target Function Representation

The representation of knowledge may be a table, collection of rules or a neural network. The linear combination of these factors can be coined as:

$$V = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_3$$

where, x_1, x_2 and x_3 represent different board features and w_0, w_1, w_2 and w_3 represent weights.

Choosing an Approximation Algorithm for the Target Function

The focus is to choose weights and fit the given training samples effectively. The aim is to reduce the error given as:

$$E \equiv \sum_{\substack{Training \\ Samples}} \left[V_{train}(b) - \hat{V}(b) \right]^2$$

Here, b is the sample and $\hat{V}(b)$ is the predicted hypothesis. The approximation is carried out as:

- Computing the error as the difference between trained and expected hypothesis. Let error be $error(b)$.
- Then, for every board feature x_i, the weights are updated as:

$$w_i = w_i + \mu \times error(b) \times x_i$$

Here, μ is the constant that moderates the size of the weight update.

Thus, the learning system has the following components:

- A Performance system to allow the game to play against itself.
- A Critic system to generate the samples.
- A Generalizer system to generate a hypothesis based on samples.
- An Experimenter system to generate a new system based on the currently learnt function. This is sent as input to the performance system.

3.4 INTRODUCTION TO CONCEPT LEARNING

Concept learning is a learning strategy of acquiring abstract knowledge or inferring a general concept or deriving a category from the given training samples. It is a process of abstraction and generalization from the data.

Concept learning helps to classify an object that has a set of common, relevant features. Thus, it helps a learner compare and contrast categories based on the similarity and association of positive and negative instances in the training data to classify an object. The learner tries to simplify by observing the common features from the training samples and then apply this simplified model to the future samples. This task is also known as learning from experience.

Each concept or category obtained by learning is a Boolean valued function which takes a true or false value. For example, humans can identify different kinds of animals based on common relevant features and categorize all animals based on specific sets of features. The special features that distinguish one animal from another can be called as a concept. This way of learning categories for object and to recognize new instances of those categories is called as concept learning. It is formally defined as inferring a Boolean valued function by processing training instances.

Concept learning requires three things:

1. Input – Training dataset which is a set of training instances, each labeled with the name of a concept or category to which it belongs. Use this past experience to train and build the model.
2. Output – Target concept or Target function f. It is a mapping function $f(x)$ from input x to output y. It is to determine the specific features or common features to identify an object. In other words, it is to find the hypothesis to determine the target concept. For e.g., the specific set of features to identify an elephant from all animals.
3. Test – New instances to test the learned model.

Formally, Concept learning is defined as–"Given a set of hypotheses, the learner searches through the hypothesis space to identify the best hypothesis that matches the target concept".

Consider the following set of training instances shown in Table 3.1.

Table 3.1: Sample Training Instances

S.No.	Horns	Tail	Tusks	Paws	Fur	Color	Hooves	Size	Elephant
1.	No	Short	Yes	No	No	Black	No	Big	Yes
2.	Yes	Short	No	No	No	Brown	Yes	Medium	No
3.	No	Short	Yes	No	No	Black	No	Medium	Yes
4.	No	Long	No	Yes	Yes	White	No	Medium	No
5.	No	Short	Yes	Yes	Yes	Black	No	Big	Yes

Here, in this set of training instances, the independent attributes considered are 'Horns', 'Tail', 'Tusks', 'Paws', 'Fur', 'Color', 'Hooves' and 'Size'. The dependent attribute is 'Elephant'. The target concept is to identify the animal to be an Elephant.

Let us now take this example and understand further the concept of hypothesis.

Target Concept: Predict the type of animal - For example –'Elephant'.

3.4.1 Representation of a Hypothesis

A *hypothesis* '*h*' approximates a target function '*f*' to represent the relationship between the independent attributes and the dependent attribute of the training instances. The hypothesis is the predicted approximate model that best maps the inputs to outputs. Each hypothesis is represented as a conjunction of attribute conditions in the antecedent part.

For example, (Tail = Short) ∧ (Color = Black)....

The set of hypothesis in the search space is called as hypotheses. Hypotheses are the plural form of hypothesis. Generally '*H*' is used to represent the hypotheses and '*h*' is used to represent a candidate hypothesis.

Each attribute condition is the constraint on the attribute which is represented as attribute-value pair. In the antecedent of an attribute condition of a hypothesis, each attribute can take value as either '?' or 'φ' or can hold a single value.

- "?" denotes that the attribute can take any value [e.g., Color = ?]
- "φ" denotes that the attribute cannot take any value, i.e., it represents a null value [e.g., Horns = φ]
- Single value denotes a specific single value from acceptable values of the attribute, i.e., the attribute 'Tail' can take a value as 'short' [e.g., Tail = Short]

For example, a hypothesis '*h*' will look like,

	Horns	Tail	Tusks	Paws	Fur	Color	Hooves	Size
h =	<No	?	Yes	?	?	Black	No	Medium>

Given a test instance x, we say $h(x) = 1$, if the test instance x satisfies this hypothesis h.

The training dataset given above has 5 training instances with 8 independent attributes and one dependent attribute. Here, the different hypotheses that can be predicted for the target concept are,

	Horns	Tail	Tusks	Paws	Fur	Color	Hooves	Size
h =	<No	?	Yes	?	?	Black	No	Medium>
				(or)				
h =	<No	?	Yes	?	?	Black	No	Big>

The task is to predict the best hypothesis for the target concept (an elephant). The most general hypothesis can allow any value for each of the attribute.

It is represented as:

<?, ?, ?, ?, ?, ?, ?, ?>. This hypothesis indicates that any animal can be an elephant.

The most specific hypothesis will not allow any value for each of the attribute <φ, φ, φ, φ, φ, φ, φ, φ>. This hypothesis indicates that no animal can be an elephant.

The target concept mentioned in this example is to identify the conjunction of specific features from the training instances to correctly identify an elephant.

Example 3.1: Explain Concept Learning Task of an Elephant from the dataset given in Table 3.1.

Given,

Input: 5 instances each with 8 attributes

Target concept/function 'c': Elephant → {Yes, No}

Hypotheses H: Set of hypothesis each with conjunctions of literals as propositions [i.e., each literal is represented as an attribute-value pair]

Solution: The hypothesis 'h' for the concept learning task of an Elephant is given as:

h = <No Short Yes ? ? Black No ?>

This hypothesis h is expressed in propositional logic form as below:

(Horns = No) ∧ (Tail = Short) ∧ (Tusks = Yes) ∧ (Paws = ?) ∧ (Fur = ?) ∧ (Color = Black) ∧ (Hooves = No) ∧ (Size = ?)

Output: Learn the hypothesis 'h' to predict an 'Elephant' such that for a given test instance x,

$$h(x) = c(x)$$

This hypothesis produced is also called as concept description which is a model that can be used to classify subsequent instances.

Thus, concept learning can also be called as *Inductive Learning* that tries to induce a general function from specific training instances. This way of learning a hypothesis that can produce an approximate target function with a sufficiently large set of training instances can also approximately classify other unobserved instances and is called as inductive learning hypothesis. We can only determine an approximate target function because it is very difficult to find an exact target function with the observed training instances. That is why a hypothesis is an approximate target function that best maps the inputs to outputs.

3.4.2 Hypothesis Space

Hypothesis space is the set of all possible hypotheses that approximates the target function f. In other words, the set of all possible approximations of the target function can be defined as hypothesis space. From this set of hypotheses in the hypothesis space, a machine learning algorithm would determine the best possible hypothesis that would best describe the target function or best fit the outputs. Generally, a hypothesis representation language represents a larger hypothesis space. Every machine learning algorithm would represent the hypothesis space in a different manner about the function that maps the input variables to output variables. For example, a regression algorithm represents the hypothesis space as a linear function whereas a decision tree algorithm represents the hypothesis space as a tree.

The set of hypotheses that can be generated by a learning algorithm can be further reduced by specifying a language bias.

The subset of hypothesis space that is consistent with all-observed training instances is called as **Version Space.** Version space represents the only hypotheses that are used for the classification.

For example, each of the attribute given in the Table 3.1 has the following possible set of values.

Horns - Yes, No

Tail - Long, Short

Tusks - Yes, No

Paws - Yes, No

Fur - Yes, No

Color - Brown, Black, White

Hooves - Yes, No

Size - Medium, Big

Considering these values for each of the attribute, there are $(2 \times 2 \times 2 \times 2 \times 2 \times 3 \times 2 \times 2) = 384$ distinct instances covering all the 5 instances in the training dataset.

So, we can generate $(4 \times 4 \times 4 \times 4 \times 4 \times 5 \times 4 \times 4) = 81{,}920$ distinct hypotheses when including two more values [?, φ] for each of the attribute. However, any hypothesis containing one or more φ symbols represents the empty set of instances; that is, it classifies every instance as negative instance. Therefore, there will be $(3 \times 3 \times 3 \times 3 \times 3 \times 4 \times 3 \times 3 + 1) = 8{,}749$ distinct hypotheses by including only '?' for each of the attribute and one hypothesis representing the empty set of instances. Thus, the hypothesis space is much larger and hence we need efficient learning algorithms to search for the best hypothesis from the set of hypotheses.

Hypothesis ordering is also important wherein the hypotheses are ordered from the most specific one to the most general one in order to restrict searching the hypothesis space exhaustively.

3.4.3 Heuristic Space Search

Heuristic search is a search strategy that finds an optimized hypothesis/solution to a problem by iteratively improving the hypothesis/solution based on a given heuristic function or a cost measure. Heuristic search methods will generate a possible hypothesis that can be a solution in

the hypothesis space or a path from the initial state. This hypothesis will be tested with the target function or the goal state to see if it is a real solution. If the tested hypothesis is a real solution, then it will be selected. This method generally increases the efficiency because it is guaranteed to find a better hypothesis but may not be the best hypothesis. It is useful for solving tough problems which could not solved by any other method. The typical example problem solved by heuristic search is the travelling salesman problem.

Several commonly used heuristic search methods are hill climbing methods, constraint satisfaction problems, best-first search, simulated-annealing, A* algorithm, and genetic algorithms.

3.4.4 Generalization and Specialization

In order to understand about how we construct this concept hierarchy, let us apply this general principle of generalization/specialization relation. By generalization of the most specific hypothesis and by specialization of the most general hypothesis, the hypothesis space can be searched for an approximate hypothesis that matches all positive instances but does not match any negative instance.

Searching the Hypothesis Space

There are two ways of learning the hypothesis, consistent with all training instances from the large hypothesis space.

1. Specialization – General to Specific learning
2. Generalization – Specific to General learning

Scan for information on *'Additional Examples on Generalization and Specialization'*

Generalization – Specific to General Learning This learning methodology will search through the hypothesis space for an approximate hypothesis by generalizing the most specific hypothesis.

Example 3.2: Consider the training instances shown in Table 3.1 and illustrate Specific to General Learning.

Solution: We will start from all false or the most specific hypothesis to determine the most restrictive specialization. Consider only the positive instances and generalize the most specific hypothesis. Ignore the negative instances.

This learning is illustrated as follows:

The most specific hypothesis is taken now, which will not classify any instance to true.

$h =$ <φ φ φ φ φ φ φ φ>

Read the first instance $I1$, to generalize the hypothesis h so that this positive instance can be classified by the hypothesis $h1$.

$I1$:	No	Short	Yes	No	No	Black	No	Big	**Yes (Positive instance)**
$h1$ =	<No	Short	Yes	No	No	Black	No	Big>	

When reading the second instance *I*2, it is a negative instance, so ignore it.

*I*2:	Yes	Short	No	No	No	Brown	Yes	Medium	**No (Negative instance)**
*h*2 = <No	Short	Yes	No	No	Black	No	Big>		

Similarly, when reading the third instance *I*3, it is a positive instance so generalize *h*2 to *h*3 to accommodate it. The resulting *h*3 is generalized.

*I*3:	No	Short	Yes	No	No	Black	No	Medium	**Yes (Positive instance)**
*h*3 = <No	Short	Yes	No	No	Black	No	?>		

Ignore *I*4 since it is a negative instance.

*I*4:	No	Long	No	Yes	Yes	White	No	Medium	**No (Negative instance)**
*h*4 = <No	Short	Yes	No	No	Black	No	?>		

When reading the fifth instance *I*5, *h*4 is further generalized to *h*5.

*I*5:	No	Short	Yes	Yes	Yes	Black	No	Big	**Yes (Positive instance)**
*h*5 = <No	Short	Yes	?	?	Black	No	?>		

Now, after observing all the positive instances, an approximate hypothesis *h*5 is generated which can now classify any subsequent positive instance to true.

Specialization – General to Specific Learning This learning methodology will search through the hypothesis space for an approximate hypothesis by specializing the most general hypothesis.

Example 3.3: Illustrate learning by Specialization – General to Specific Learning for the data instances shown in Table 3.1.

Solution: Start from the most general hypothesis which will make true all positive and negative instances.

Initially,

h = <?	?	?	?	?	?	?	?>	

h is more general to classify all instances to true.

*I*1:	No	Short	Yes	No	No	Black	No	Big	**Yes (Positive instance)**
*h*1 = <?	?	?	?	?	?	?	?>		

*I*2 :	Yes	Short	No	No	No	Brown	Yes	Medium	**No (Negative instance)**
*h*2 = <No	?	?	?	?	?	?	?>		
	<?	?	Yes	?	?	?	?	?>	
	<?	?	?	?	?	Black	?	?>	
	<?	?	?	?	?	?	No	?>	
	<?	?	?	?	?	?	?	Big>	

*h*2 imposes constraints so that it will not classify a negative instance to true.

*I*3:	No	Short	Yes	No	No	Black	No	Medium	**Yes (Positive instance)**
*h*3 = <No	?	?	?	?	?	?	?>		
	<?	?	Yes	?	?	?	?	?>	

	<?	?	?	?	?	Black	?	?>
	<?	?	?	?	?	?	No	?>
	<?	?	?	?	?	?	?	Big>
*I*4:	No	Long	No	Yes	Yes	White	No	Medium **No (Negative instance)**
*h*4 =	<?	?	Yes	?	?	?	?	?>
	<?	?	?	?	?	Black	?	?>
	<?	?	?	?	?	?	?	Big>

Remove any hypothesis inconsistent with this negative instance.

*I*5:	No	Short	Yes	Yes	Yes	Black	No	Big **Yes (Positive instance)**
*h*5 =	<?	?	Yes	?	?	?	?	?>
	<?	?	?	?	?	Black	?	?>
	<?	?	?	?	?	?	?	Big>

Thus, *h*5 is the hypothesis space generated which will classify the positive instances to true and negative instances to false.

3.4.5 Hypothesis Space Search by Find-S Algorithm

Find-S algorithm is guaranteed to converge to the most specific hypothesis in *H* that is consistent with the positive instances in the training dataset. Obviously, it will also be consistent with the negative instances. Thus, this algorithm considers only the positive instances and eliminates negative instances while generating the hypothesis. It initially starts with the most specific hypothesis.

Algorithm 3.1: Find-S

Input: Positive instances in the Training dataset

Output: Hypothesis *'h'*

1. Initialize *'h'* to the most specific hypothesis.

   $h = <\varphi \quad \varphi \quad \varphi \quad \varphi \quad \varphi \quad .....>$

2. Generalize the initial hypothesis for the first positive instance [Since *'h'* is more specific].
3. For each subsequent instances:

   If it is a positive instance,

   Check for each attribute value in the instance with the hypothesis *'h'*.

   If the attribute value is the same as the hypothesis value, then do nothing,

   Else if the attribute value is different than the hypothesis value, change it to '?' in *'h'*.

   Else if it is a negative instance,

   Ignore it.

Example 3.4: Consider the training dataset of 4 instances shown in Table 3.2. It contains the details of the performance of students and their likelihood of getting a job offer or not in their final semester. Apply the Find-S algorithm.

Table 3.2: Training Dataset

CGPA	Interactiveness	Practical Knowledge	Communication Skills	Logical Thinking	Interest	Job Offer
≥9	Yes	Excellent	Good	Fast	Yes	Yes
≥9	Yes	Good	Good	Fast	Yes	Yes
≥8	No	Good	Good	Fast	No	No
≥9	Yes	Good	Good	Slow	No	Yes

Solution:

Step 1: Initialize '*h*' to the most specific hypothesis. There are 6 attributes, so for each attribute, we initially fill 'φ' in the initial hypothesis '*h*'.

$h = <\varphi \quad \varphi \quad \varphi \quad \varphi \quad \varphi \quad \varphi>$

Step 2: Generalize the initial hypothesis for the first positive instance. *I*1 is a positive instance, so generalize the most specific hypothesis '*h*' to include this positive instance. Hence,

*I*1: ≥9 Yes Excellent Good Fast Yes **Positive instance**

h = <≥9 Yes Excellent Good Fast Yes>

Step 3: Scan the next instance *I*2, since *I*2 is a positive instance. Generalize '*h*' to include positive instance *I*2. For each of the non-matching attribute value in '*h*' put a '?' to include this positive instance. The third attribute value is mismatching in '*h*' with *I*2, so put a '?'.

*I*2: ≥9 Yes Good Good Fast Yes **Positive instance**

h = <≥9 Yes ? Good Fast Yes>

Now,scan *I*3. Since it is a negative instance, ignore it. Hence, the hypothesis remains the same without any change after scanning *I*3.

I3: ≥8 No Good Good Fast No **Negative instance**

h = <≥9 Yes ? Good Fast Yes>

Now scan *I*4. Since it is a positive instance, check for mismatch in the hypothesis '*h*' with *I*4. The 5th and 6th attribute value are mismatching, so add '?' to those attributes in '*h*'.

I4: ≥9 Yes Good Good Slow No **Positive instance**

h = <≥9 Yes ? Good ? ?>

Now, the final hypothesis generated with Find-S algorithm is:

h = <≥9 Yes ? Good ? ?>

It includes all positive instances and obviously ignores any negative instance.

Limitations of Find-S Algorithm

1. Find-S algorithm tries to find a hypothesis that is consistent with positive instances, ignoring all negative instances. As long as the training dataset is consistent, the hypothesis found by this algorithm may be consistent.
2. The algorithm finds only one unique hypothesis, wherein there may be many other hypotheses that are consistent with the training dataset.

3. Many times, the training dataset may contain some errors; hence such inconsistent data instances can mislead this algorithm in determining the consistent hypothesis since it ignores negative instances.

Hence, it is necessary to find the set of hypotheses that are consistent with the training data including the negative examples. To overcome the limitations of Find-S algorithm, Candidate Elimination algorithm was proposed to output the set of all hypotheses consistent with the training dataset.

3.4.6 Version Spaces

The version space contains the subset of hypotheses from the hypothesis space that is consistent with all training instances in the training dataset.

Scan for information on *'Additional Examples on Version Spaces'*

List-Then-Eliminate Algorithm

The principle idea of this learning algorithm is to initialize the version space to contain all hypotheses and then eliminate any hypothesis that is found inconsistent with any training instances. Initially, the algorithm starts with a version space to contain all hypotheses scanning each training instance. The hypotheses that are inconsistent with the training instance are eliminated. Finally, the algorithm outputs the list of remaining hypotheses that are all consistent.

Algorithm 3.2: List-Then-Eliminate

Input: Version Space – a list of all hypotheses

Output: Set of consistent hypotheses

1. Initialize the version space with a list of hypotheses.
2. For each training instance,
   - remove from version space any hypothesis that is inconsistent.

This algorithm works fine if the hypothesis space is finite but practically it is difficult to deploy this algorithm. Hence, a variation of this idea is introduced in the Candidate Elimination algorithm.

Version Spaces and the Candidate Elimination Algorithm

Version space learning is to generate all consistent hypotheses around. This algorithm computes the version space by the combination of the two cases namely,

- Specific to General learning – Generalize S to include the positive example
- General to Specific learning – Specialize G to exclude the negative example

Using the Candidate Elimination algorithm, we can compute the version space containing all (and only those) hypotheses from H that are consistent with the given observed sequence of training instances. The algorithm defines two boundaries called *'general boundary'* which is a set of all hypotheses that are the most general and *'specific boundary'* which is a set of all hypotheses that are the most specific. Thus, the algorithm limits the version space to contain only those hypotheses that are most general and most specific. Thus, it provides a compact representation of List-then algorithm.

Algorithm 3.3: Candidate Elimination

Input: Set of instances in the Training dataset

Output: Hypothesis *G* and *S*

1. Initialize *G*, to the maximally general hypotheses.
2. Initialize *S*, to the maximally specific hypotheses.
   - Generalize the initial hypothesis for the first positive instance.
3. For each subsequent new training instance,
   - If the instance is **positive**,
     - Generalize *S* to include the positive instance,
       - Check the attribute value of the positive instance and *S*,
         - If the attribute value of positive instance and *S* are different, fill that field value with '?'.
         - If the attribute value of positive instance and *S* are same, then do no change.
     - Prune *G* to exclude all inconsistent hypotheses in *G* with the positive instance.
   - If the instance is **negative**,
     - Specialize *G* to exclude the negative instance,
       - Add to *G* all minimal specializations to exclude the negative example and be consistent with *S*.
         - If the attribute value of *S* and the negative instance are different, then fill that attribute value with *S* value.
         - If the attribute value of *S* and negative instance are same, no need to update '*G*' and fill that attribute value with '?'.
     - Remove from *S* all inconsistent hypotheses with the negative instance.

Generating Positive Hypothesis 'S' If it is a positive example, refine *S* to include the positive instance. We need to generalize *S* to include the positive instance. The hypothesis is the conjunction of '*S*' and positive instance. When generalizing, for the first positive instance, add to *S* all minimal generalizations such that *S* is filled with attribute values of the positive instance. For the subsequent positive instances scanned, check the attribute value of the positive instance and *S* obtained in the

previous iteration. If the attribute values of positive instance and *S* are different, fill that field value with a '?'. If the attribute values of positive instance and *S* are same, no change is required.

If it is a negative instance, it skips.

Generating Negative Hypothesis 'G' If it is a negative instance, refine *G* to exclude the negative instance. Then, prune *G* to exclude all inconsistent hypotheses in *G* with the positive instance. The idea is to add to *G* all minimal specializations to exclude the negative instance and be consistent with the positive instance. Negative hypothesis indicates general hypothesis.

If the attribute values of positive and negative instances are different, then fill that field with positive instance value so that the hypothesis does not classify that negative instance as true. If the attribute values of positive and negative instances are same, then no need to update '*G*' and fill that attribute value with a '?'.

Generating Version Space – [Consistent Hypothesis] We need to take the combination of sets in '*G*' and check that with '*S*'. When the combined set fields are matched with fields in '*S*', then only that is included in the version space as consistent hypothesis.

Example 3.4: Consider the same set of instances from the training dataset shown in Table 3.3 and generate version space as consistent hypothesis.

Solution:

Step 1: Initialize '*G*' boundary to the maximally general hypotheses,

G = <? ? ? ? ? ?>

Step 2: Initialize '*S*' boundary to the maximally specific hypothesis. There are 6 attributes, so for each attribute, we initially fill 'φ' in the hypothesis '*S*'.

S = <φ φ φ φ φ φ>

Generalize the initial hypothesis for the first positive instance. *I*1 is a positive instance; so generalize the most specific hypothesis '*S*' to include this positive instance. Hence,

*I*1:	≥9	Yes	Excellent	Good	Fast	Yes	**Positive instance**
*S*1 = <	≥9	Yes	Excellent	Good	Fast	Yes>	
*G*1 = <?	?	?		?	?	?>	

Step 3:

Iteration 1

Scan the next instance *I*2. Since *I*2 is a positive instance, generalize '*S*1' to include positive instance *I*2. For each of the non-matching attribute value in '*S*1', put a '?' to include this positive instance. The third attribute value is mismatching in '*S*1' with *I*2, so put a '?'.

*I*2:	≥9	Yes	Good	Good	Fast	Yes	**Positive instance**
*S*2 = <	≥9	Yes	?	Good	Fast	Yes>	

Prune *G*1 to exclude all inconsistent hypotheses with the positive instance. Since *G*1 is consistent with this positive instance, there is no change. The resulting *G*2 is,

*G*2 = <? ? ? ? ? ?>

Iteration 2

Now Scan *I*3,

*I*3:	≥8	No	Good	Good	Fast	No	**Negative instance**

Since it is a negative instance, specialize *G*2 to exclude the negative example but stay consistent with *S*2. Generate hypothesis for each of the non-matching attribute value in *S*2 and fill with the attribute value of *S*2. In those generated hypotheses, for all matching attribute values, put a '?'. The first, second and 6th attribute values do not match, hence '3' hypotheses are generated in *G*3.

There is no inconsistent hypothesis in *S*2 with the negative instance, hence *S*3 remains the same.

*G*3 = <	≥9	?	?	?	?	?>
	<?	Yes	?	?	?	?>
	<?	?	?	?	?	Yes>
*S*3 = <	≥9	Yes	?	Good	Fast	Yes>

Iteration 3

Now Scan *I*4. Since it is a positive instance, check for mismatch in the hypothesis '*S*3' with *I*4. The 5th and 6th attribute value are mismatching, so add '?' to those attributes in '*S*4'.

*I*4:	≥9	Yes	Good	Good	Slow	No **Positive instance**
*S*4 = <	≥9	Yes	?	Good	?	?>

Prune *G*3 to exclude all inconsistent hypotheses with the positive instance *I*4.

*G*3 = <	≥9	?	?	?	?	?>
	<?	Yes	?	?	?	?>
	<?	?	?	?	?	Yes> **Inconsistent**

Since the third hypothesis in *G*3 is inconsistent with this positive instance, remove the third one. The resulting *G*4 is,

*G*4 = <	≥9	?	?	?	?	?>
	<?	Yes	?	?	?	?>

Using the two boundary sets, *S*4 and *G*4, the version space is converged to contain the set of consistent hypotheses.

The final version space is,

<	≥9	Yes	?	?	?	?>
<	≥9	?	?	Good	?	?>
	<?	Yes	?	Good	?	?>

Thus, the algorithm finds the version space to contain only those hypotheses that are most general and most specific.

The diagrammatic representation of deriving the version space is shown in Figure 3.2.

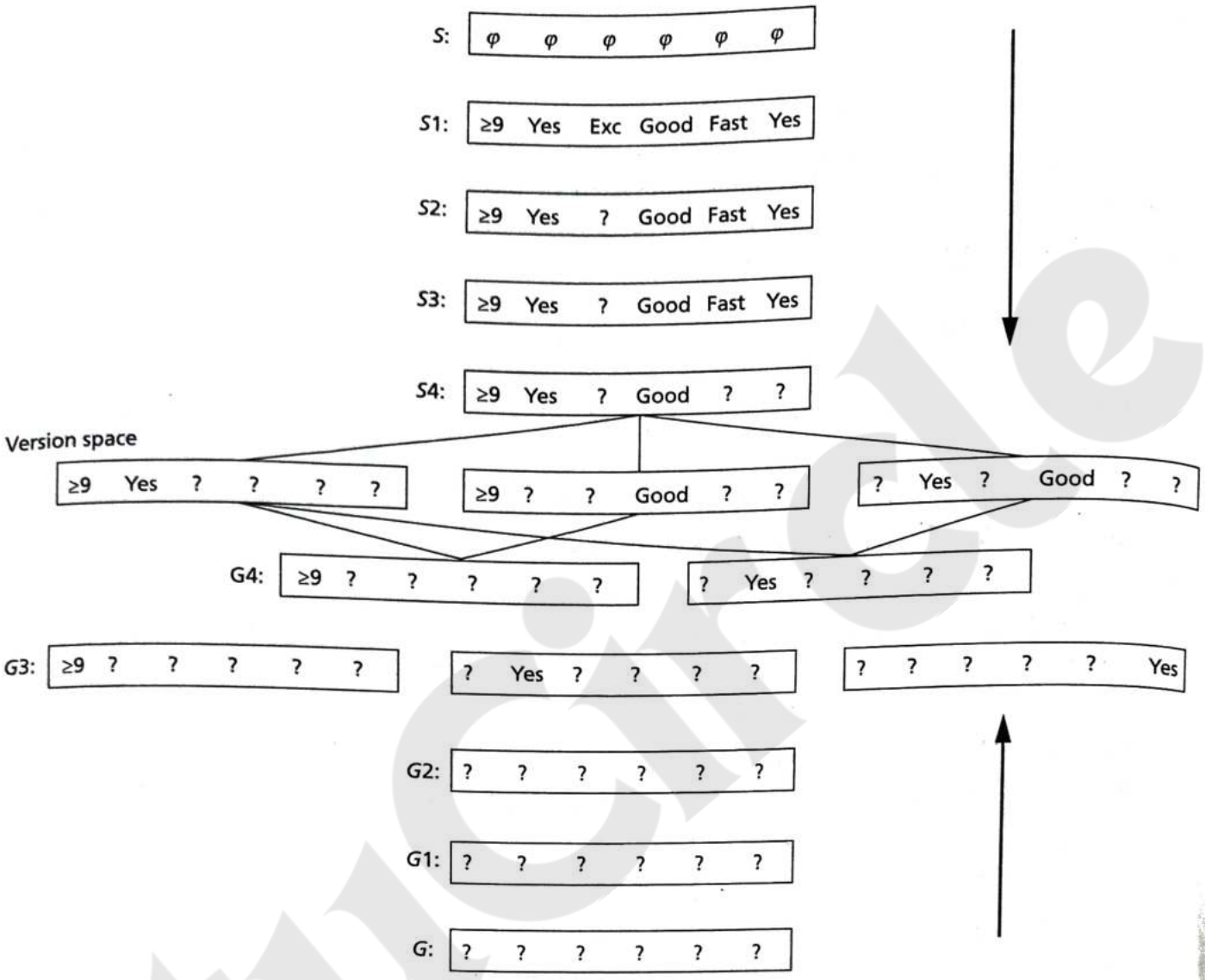

Figure 3.2: Deriving the Version Space

VtuCircle

3.6 MODELLING IN MACHINE LEARNING

A machine learning model is an abstraction of the training dataset that can perform a prediction on new data. Training the model means feeding instances to the machine learning algorithm. Training datasets are used to fit and tune the model. After training a machine learning algorithm with the training data, a predictive model is generated as output to which a new data is fed to make predictions.

The process of modelling means training a machine learning algorithm with the training dataset, tuning it to increase performance, validating it and making predictions for a new unseen data. The major concern in machine learning is what model to select, how to train the model, time required to train, the dataset to be used, what performance to expect, and so on.

Learning the parameters is the main goal in machine learning algorithms. There are two types of parameters – model parameters and hyperparameters. Certain parameters can be learnt directly from training data and are called model parameters. For example, the coefficients used in

regression model, split attributes in decision tree model, weights and biases in neural networks and so on. Hyperparameters are higher-level parameters which cannot be learnt directly. For example, regularization lambda λ used in regularized regression, number of decision trees to include in a random forest, and so on.

Evaluating the selected machine learning model is also equally important as training the model. Hence, the dataset is split into two subsets called training dataset and test dataset, wherein the training dataset is used to train the model and the test dataset is used to evaluate the model. During training, the test dataset is unseen to the model so that the model can be tested properly on its ability to predict a new data. If the training and test datasets are the same, then the model can overfit but it would perform poorly when given a new unseen data.

During prediction, an error occurs when the estimated output does not match with the true output. Training error, also called as in-sample error, results when applying the predicted model on the training data, while Test error also called as out-of-sample error is the average error when predicting on unseen observations. The error function or the loss function is the aggregation of the differences between the true values and the predicted values. This loss function is defined as the **Mean Squared Error** (MSE), which is the average of the squared differences between the true values Y_i and the predicted values $f(X_i)$ for an input value 'X_i'. A smaller value of MSE denotes that the error is less and, therefore, the prediction is more accurate.

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}[Y_i - f(X_i)]^2$$

Machine Learning Process

The four basic steps in the machine learning process are:

1. Choose a machine learning algorithm to suit the training data and the problem domain
2. Input the training dataset and train the machine learning algorithm to learn from the data and capture the patterns in the data
3. Tune the parameters of the model to improve the accuracy of learning of the algorithm
4. Evaluate the learned model once the model is built

3.6.1 Model Selection and Model Evaluation

The biggest challenge in machine learning is choosing an algorithm that suits the problem. Hence, model selection and assessment are very important and deal with two types of complexities.

1. Model Performance – How well the model performs on the training dataset?
2. Model Complexity – How much complexity the model possesses after the training phase is over?

Model Selection is a process of selecting one good enough model among different machine learning models for the dataset or selecting different sets of features or hyperparameters for the same machine learning model. It is difficult to find the best model because all models exhibit some predictive error for the problem, so atleast a good enough model should be selected that performs fairly well with the dataset.

Some of the approaches used for selecting a machine learning model are listed below:

1. Use resample methods and split the dataset as training, testing and validation datasets and observe the performance of the model over all the phases. This approach is suitable for smaller datasets.
2. The simplest approach is to fit a model on the training dataset and to compute measures like error or accuracy.
3. The use of probabilistic framework and quantification of the performance of the model as a score is the third approach.

These methods are discussed in the following sections.

3.6.2 Re-sampling Methods

Re-sampling is a technique to select a model by reconstructing the training dataset and test dataset by randomly choosing instances by some method from the given dataset. This method involves selecting different instances repeatedly from a training dataset to tune a model. It is done to improve the accuracy of a model. The common re-sampling model selection methods are Random train/test splits, Cross-Validation (*K*-fold, LOOCV, etc.) and Bootstrap.

Cross-Validation

Cross-Validation is a method by which we can tune the model with only training dataset. It is a model evaluation approach by which we can set aside some data of the training dataset for validation and fit the rest of the data to train the model. The best model is found by estimating the average of errors on different test data. The popular cross-validation family of methods includes Holdout method, *K*-fold cross-validation, Stratified cross-validation and Leave-One-Out Cross-Validation (LOOCV).

Holdout Method

This is the simplest method of cross-validation. The dataset is split into two subsets called training dataset and test dataset. The model is trained using the training dataset and then evaluated using the test dataset. This holdout method can be applied for a single time which is called as single holdout method or it can be repeated for more than once which is called as repeated holdout method. The average performance on the test dataset is estimated to evaluate the model. Even though this model is very simple, it can exhibit high variance and the performance largely depends on how the dataset is split.

K-fold Cross-Validation

Another way of cross-validating is using a *k*-fold cross-validation, which will split the training dataset into k equal folds/parts creating $k - 1$ subsets of training set and one test subset. Out of the k folds, $k - 1$ folds are used for training and one fold is used for testing the model. This has to be performed for k iterations and during each iteration a different fold is selected for testing. The average performance of the model on k iterations is the final estimate of the model performance.

The illustration of this re-sampling is shown in Figure 3.4.

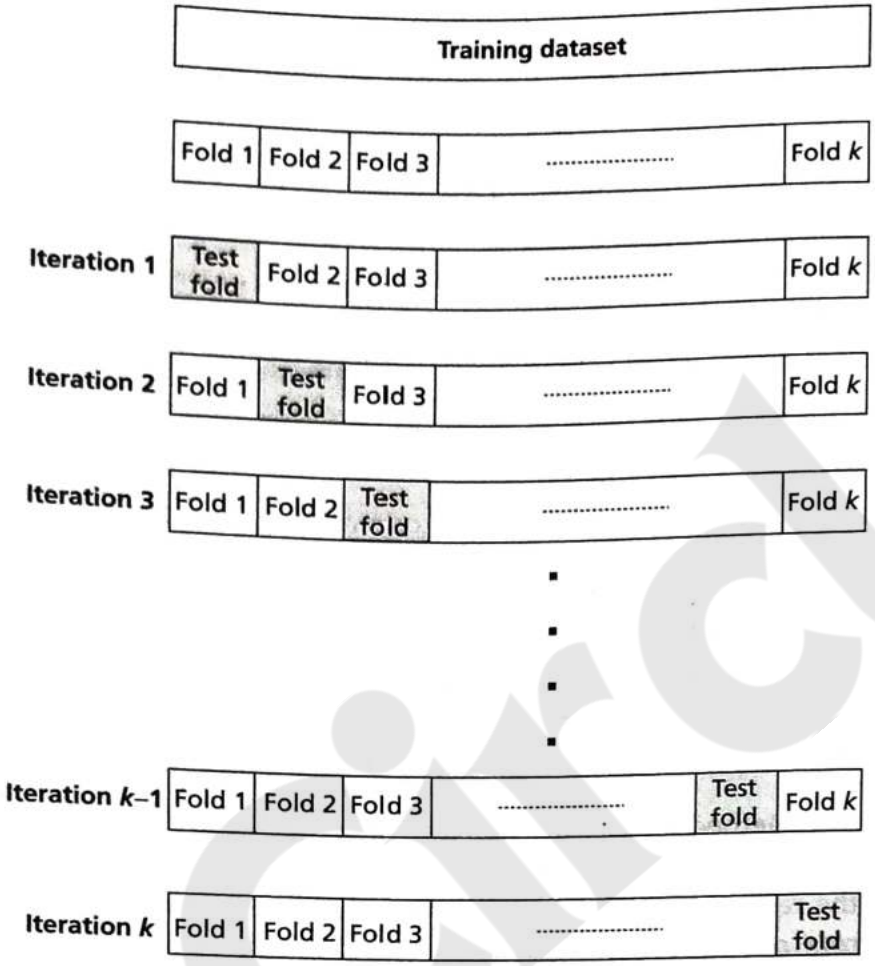

Figure 3.4: Illustration of *K*-fold Cross-Validation

Algorithm 3.4: *K*-fold Cross Validation

Input: Dataset with '*n*' instances

'*k*' – Number of folds

1. Repeat the following steps for '*k*' times with a different 'hold-out' fold for evaluation:
   - Split the dataset into *k* equal folds.
   - Train the model on $(k-1)$ folds.
   - Evaluate it on the 1 remaining "hold-out" fold.
2. Compute average of the performance across all *k* hold-out folds.

Stratified K-fold Cross-Validation

This method is similar to *k*-fold cross-validation but with a slight difference. Here, it is ensured that while splitting the dataset into *k* folds, each fold should contain the same proportion of instances with a given categorical value. This is called stratified cross-validation.

Leave-One-Out Cross-Validation (LOOCV)

This method repeatedly splits the n data instances of the dataset into training dataset containing $n - 1$ data instances and leaving one data instance for evaluating the model. This process is repeated n times and average test error is then estimated for the model. Even though this model is expensive and time consuming because it has to run for n times (i.e., n data instances in the dataset), it has less bias. For example, if the training dataset contains 100 data instances, then 99 instances are used for training and one instance to test or evaluate the model. This process is repeated 100 times selecting a different instance as holdout instance for testing in each iteration.

The illustration of this re-sampling is shown in Figure 3.5.

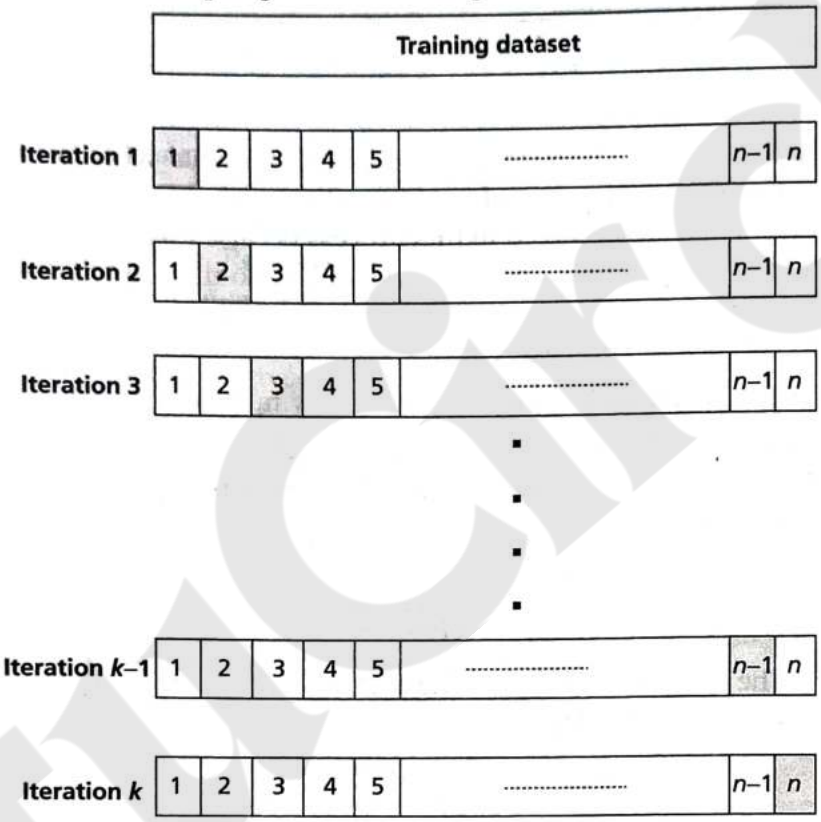

Figure 3.5: Illustration of Leave-One-Out Cross-Validation

Algorithm 3.5: LOOCV

Input Dataset with 'n' instances:

1. Repeat the following steps for 'n' times with a different 'hold-out' data instance for evaluation:
   - Train the model on $(n - 1)$ data instances.
   - Evaluate it on the one remaining 'hold-out' test instance.
2. Compute average of the performance across all n holdouts.

Model Performance

Classifier models are discussed in the subsequent chapters. The focus of this section is the evaluation of classifier models. Classifiers are unstable as a small change in the input can change the output. A solid framework is needed for proper evaluation. There are several metrics that can be used to describe the quality and usefulness of a classifier. One way to compute the metrics is to form a table called contingency table. For example, consider a test for detecting a disease, say cancer. Table 3.4 shows a contingency table for this scenario.

Table 3.4: Contingency Table

Test vs Disease	Has Disease Cancer	Has No Disease As Cancer
Positive	True Positive	False Positive
Negative	False Negative	True Negative

In this table, True Positive (TP) = Number of cancer patients who are classified by the test correctly, True Negative (TN) = Number of normal patients who do not have cancer are correctly detected. The two errors that are involved in this process is False Positive (FP) that is an alarm that indicates that the tests show positive when the patient has no disease and False Negative (FN) is another error that says a patient has cancer when tests says negative or normal. FP and FN are costly errors in this classification process.

The metrics that can be derived from this contingency table are listed below:

1. Sensitivity – The sensitivity of a test is the probability that it will produce a true positive result when used on a test dataset. It is also known as true positive rate. The sensitivity of a test can be determined by calculating:

$$\frac{TP}{TP + FN}$$

2. Specificity – The specificity of a test is the probability that a test will produce a true negative result when used on test dataset.

$$\frac{TN}{TN + FP}$$

3. Positive Predictive Value – The positive predictive value of a test is the probability that an object is classified correctly when a positive test result is observed.

$$\frac{TP}{TP + FP}$$

4. Negative Predictive Value – The negative predictive value of a test is the probability that an object is not classified properly when a negative test result is observed.

$$\frac{TN}{TN + FN}$$

5. Accuracy – The accuracy of the classifier can be shown in terms of sensitivity computed as:

$$\frac{TP + TN}{TP + TN + FP + FN}$$

6. Precision – Precision is also known as positive predictive power. It is defined as the ratio of true positive divided by the sum of true positive and false positive.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Precision indicates how good classifier is in predicting the positive classes.

7. Recall – It is same as sensitivity.

$$\text{Recall} = \text{Sensitivity} = \frac{TP}{TP + FN}$$

A combination of harmonic mean of precision and recall is called *F*-measure or *F*1 score. This is useful in identifying the model skill for a specific threshold.

Classifier Performance as Distance Measures The classifier performance can be computed as a distance measure also. The classifier accuracy can be plotted as a point. A point in the north-west is a better classifier. Euclid distance of two points of the two classifiers can give a performance measure. The value ranges from 0 to 1.

Visual Classifier Performance Receiver Operating Characteristic (ROC) curve and Precision-Recall curves indicate the performance of classifiers visually. ROC curves are visual means of checking the accuracy and comparison of classifiers. ROC is a plot of sensitivity (True Positive Rate) and the 1-specificity (False Positive Rate) for a given model.

A sample ROC curve is shown in Figure 3.6, where results of five classifiers are given. A is the ROC of an average classifier. The ideal classifier is E where the area under curve is 1.0. Theoretically, it can range from 0.9 to 1. The rest of the classifiers B, C, D are categorized based on area under curve as good, better and still better based on the area under curve values.

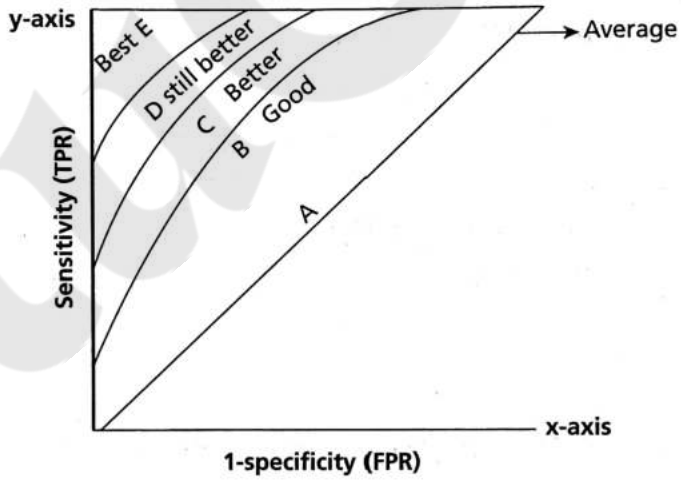

Figure 3.6: A Sample ROC Curve

The classifier prediction is based on the threshold value. A threshold value of 0.5 maps the probability to class 0 or class 1. The threshold value can be tuned to control the higher or lower FP and FN. This is useful when one focuses on error – FP or FN for an experiment.

We start from the bottom left-hand corner initially. If we have any true positive case, we move up and plot a point. If it is a false positive case, we move right and plot. This process is repeated until the complete curve is drawn. In ROC, the diagonal of the plot indicates the model has no skill or random classifier, and skillful models show the curve above the diagonal. In short, if the ROC curve is closer to the diagonal line, then it shows the classifier to be less accurate.

Instead of predicting the label of a classifier, one can predict the probabilities of the model. Probabilities allow some better evaluation by functions that are called scoring functions or scoring rules. The area under curve (AUC) is one such score that can be used for classifier model evaluation. The integrated AUC is a measure of the model across threshold values.

AUC indicates the accuracy of the model. A model is perfect if it has area under ROC curve as one. The AUC score 0 of a model indicates the wrong model. The approximate area under precision-recall curve also indicates the power of the model across thresholds.

A precision-recall curve is a plot of precision and recall for different threshold values. This curve is useful if there is an imbalance in the classes where one class has more labels and other classes have less samples.

ROC is used when there is no class imbalance and precision-recall curves are used when there is a moderate-to-large class imbalance.

Scoring Methods

Another alternative for model selection is to combine the complexity of the model and performance of the model as a score. Then, model selection is done by selecting the model that maximizes or minimizes the score.

Minimum Description Length (MDL) is one such method. The aim is to describe target variable and model in terms of bits. MDL is the principle of using minimum number of bits to represent the data and model. It is a variant of Occom Razor's principle that states that the model with the simplest explanation is the best model. MDL too recommends the selection of the hypothesis that minimizes the sum of two descriptions of data and model.

Let h be a learning model. Let $L(h)$ is the number of bits used to represent the model and D is the number of predictions, then the MDL is given as:

$$L(h) + L(D \mid h)$$

where, $L(D \mid h)$ is the number of bits used to represent the predictions D based on the training set.

MDL can be expressed in terms of negative log-likelihood also as:

$$\text{MDL} = -\log(p(\Theta)) - \log(p(y \mid x, \Theta))$$

where, y is the target variable, x is the input and Θ is the model parameters.